Lab 6: Code and Branch Coverage Analysis

Systematic White-Box Testing, Branch Analysis, and Decision Verification

Assigned Repo:	https://github.com/SETAPESU26/L6_L7_58_restaurant_bill	**Target Coverage:**	≥ 90% (Achieved: 100% Line, 100% Branch)
Application:	restaurant_bill.py (33 statements, 22 branches)	**Deliverable Commits:**	2 Dedicated Verified Commits
Author / GitHub:	PurposeKnight (pranay)	**Tooling Stack:**	Python 3.14.7, pytest 9.2.0, coverage 7.17.0

1. Objectives & Theoretical Foundations

Code coverage measures the proportion of source code executed while running an automated test suite. High test coverage builds confidence in software reliability, detects dead code, and exposes untested logical paths. However, different coverage criteria provide vastly different levels of rigor:

Coverage Type	Formal Definition	Significance & Limitations
Line (Statement) Coverage	Measures percentage of executable code statements run at least once during testing.	Baseline metric. Weak criterion because it can report 100% while missing critical alternate decision branches.
Branch Coverage	Measures whether every control decision (if, elif, else) evaluates to both True and False outcomes.	Significantly stronger than line coverage. Guarantees that all decision paths and edge cases are validated.
Path Coverage	Measures whether every linearly independent combination of paths from entry to exit is executed.	Exhaustive verification, but exponentially complex for larger programs due to combinatorial path explosion.
Function Coverage	Measures whether every declared function/method has been invoked at least once.	Coarse-grained metric indicating basic reachability across modules.

The Fallacy of Line Coverage: Why 100% Line Coverage ≠ 100% Branch Coverage

A critical principle emphasized in this laboratory is that **100% line coverage can produce a false sense of security**. When an if statement without an explicit else evaluates to True, every statement within the block is executed, yielding 100% statement coverage. However, the False branch—the decision path where the condition is not satisfied and the block is skipped—remains completely unexercised. Branch coverage analysis (coverage run --branch) detects these gaps by recording conditional jump instructions in Python bytecode.

2. Application Under Test (restaurant_bill.py)

The application encapsulates restaurant billing, splitting, and seating management through three modular functions:

1. **bill_total(section, food_total, guests, is_happy_hour):** Validates seating section against {'dine_in': 10, 'takeaway': 0, 'rooftop': 15} (raises ValueError('unknown section')); checks food_total ≥ 0 and guests ≥ 1 (raises ValueError('invalid order')); applies 20% happy hour discount on food total (except takeaway); adds section service charge and +■200 surcharge for large parties (guests ≥ 8); adds 5% tax.
2. **split_bill(total, people, tip_percent):** Validates people ≥ 1 (raises ValueError('invalid people count')) and tip_percent ≥ 0 (raises ValueError('invalid tip')); computes grand total with tip; computes per-person share using ceiling division (adds 1 to share if remainder ≠ 0).
3. **table_wait(party_size, free_tables, has_reservation):** Directs parties > 12 to 'call ahead'; accommodates reservations immediately ('seat now'); assigns 'waitlist' if free_tables == 0 or if party_size > 6 and free_tables < 2; otherwise assigns 'seat now'.

3. Baseline Initial Coverage (Starter Test Suite)

The repository as initially cloned contained two basic starter tests in test_restaurant_bill.py: test_dine_in_bill() and test_even_split(). Running the coverage tool on this initial suite produces the baseline report shown below:

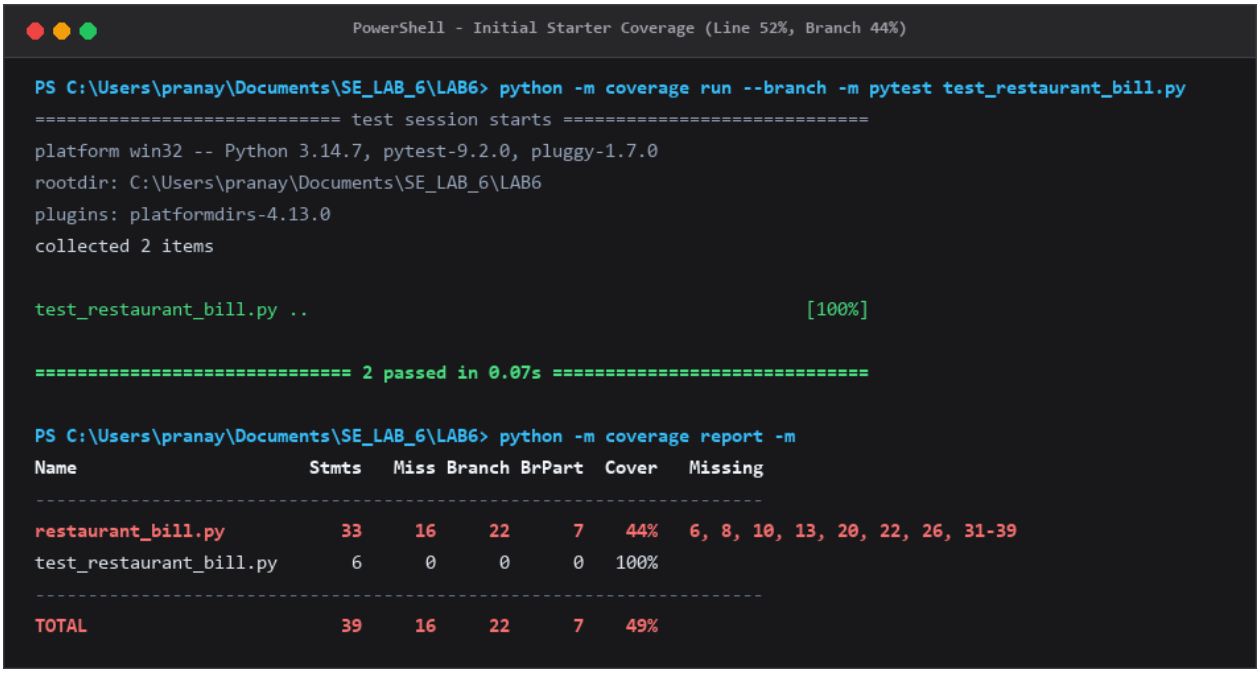


Figure 1: Baseline Initial Coverage — 52% Line Coverage, 44% Branch Coverage (16 lines and 7 branch parts missed).

Baseline Deficiencies & Untested Code Paths

An inspection of the starter coverage report reveals severe omissions across the entire codebase:

Code File / Function	Missing Lines	Untested Logical Pathway & Risk
restaurant_bill.py bill_total()	Lines 6, 8, 10, 13	Unrecognized section exception (L6), invalid order exception (L8), happy hour discount application (L10), and large group surcharge (L13) are entirely unexecuted.
restaurant_bill.py split_bill()	Lines 20, 22, 26	Invalid diner count exception (L20), negative tip exception (L22), and bill splitting with non-zero remainder ceiling division (L26) are untested.
restaurant_bill.py table_wait()	Lines 31–39 (Entire function)	The entire function is completely uncalled (0% coverage). None of the 5 seating decisions or boundary conditions are validated.

Summary of Baseline Metrics:

- **Statements Executed:** 17 / 33 (52% Line Coverage on restaurant_bill.py)
- **Decision Branches Evaluated:** Only 15 / 22 branch outcomes visited (44% Branch Coverage, 7 partial branch decisions)
- **Objective:** Advance line coverage to ≥ 90% (achieved 100%) in Deliverable 1, and branch coverage to ≥ 90% (achieved 100%) in Deliverable 2.

4. Deliverable 1: Code Coverage Analysis (CodeCoverageTest_restaurant_bill.py)

To satisfy Step 5 and Deliverable 1 of the laboratory rubric (≥ 90% code coverage requirement), a dedicated test suite CodeCoverageTest_restaurant_bill.py was engineered. The suite targets every individual statement in restaurant_bill.py across 11 targeted test cases:

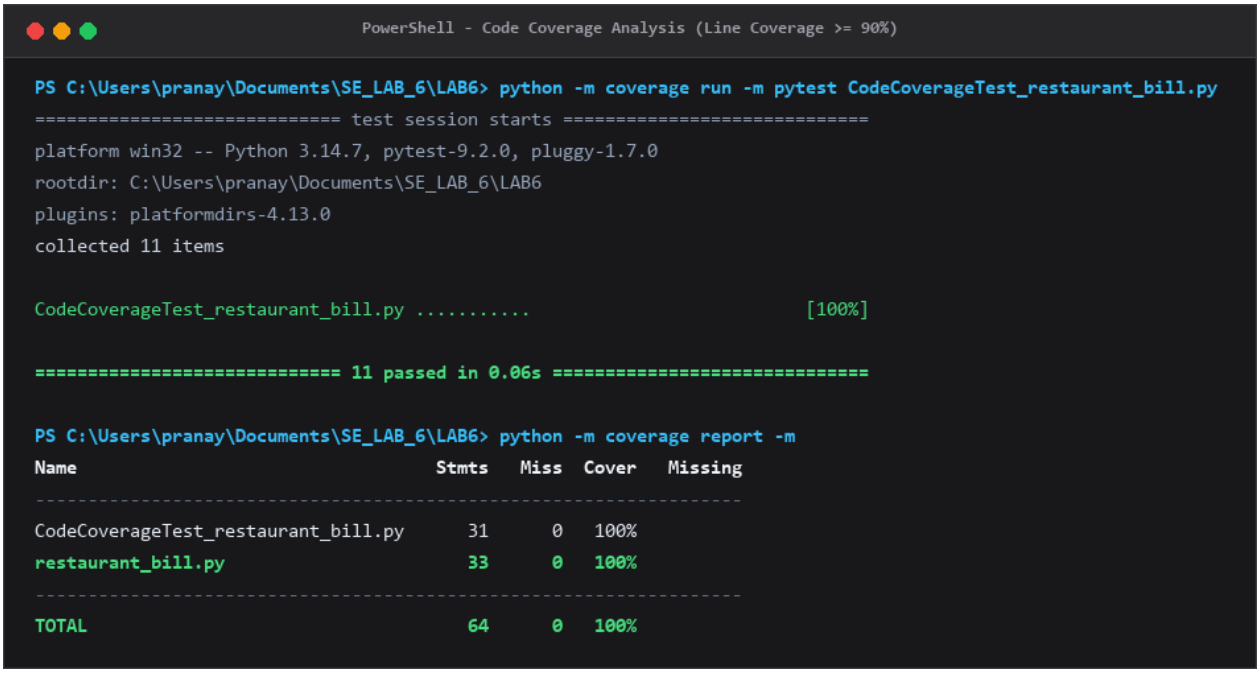


Figure 2: Statement Coverage Report — 100% Line Coverage (33/33 statements executed, 0 missed).

Test Design & Statement Target Summary

Target Function	Test Case Function Name	Target Statements & Verification Goal
bill_total()	test_bill_total_happy_hour_large_group test_bill_total_invalid_section test_bill_total_negative_food	Exercises happy hour discount (L10), large group surcharge (L13), section error handling (L6), and order validation (L8).
split_bill()	test_split_bill_invalid_people test_split_bill_invalid_tip test_split_bill_with_remainder	Exercises diner count error (L20), negative tip error (L22), tip computation (L23), integer share (L24), and remainder ceiling division (L26).
table_wait()	test_table_wait_call_ahead test_table_wait_reservation test_table_wait_no_free_tables test_table_wait_large_party_limited_tables test_table_wait_seat_now	Exercises party > 12 call ahead (L32), reservation seating (L34), zero free tables waitlist (L36), party > 6 with 1 table waitlist (L38), and standard seating fallthrough (L39).

Code Coverage Result: Executing coverage run -m pytest CodeCoverageTest_restaurant_bill.py passed all 11 test items in 0.06s and produced **100% statement coverage** (0 missed statements out of 33). This exceeds the lab's 90% benchmark.

5. Branch Gap Analysis of Code Coverage Test Suite

In accordance with the pedagogical objective outlined on Page 3 of the lab handout, CodeCoverageTest_restaurant_bill.py was evaluated under branch coverage mode (coverage run --branch -m pytest CodeCoverageTest_restaurant_bill.py). Despite achieving 100% line coverage, branch analysis reveals that **3 critical decision branches were never taken**:

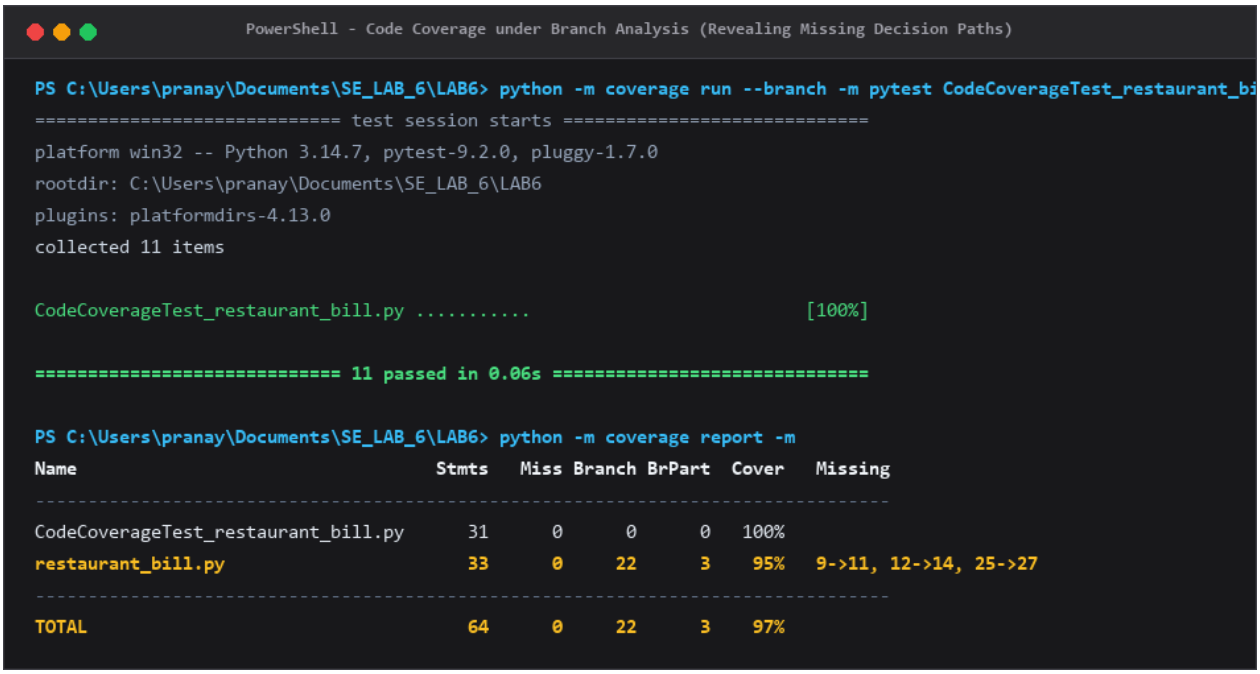


Figure 3: Branch Analysis of CodeCoverageTest — Exposing unexercised branch jumps 9->11, 12->14, and 25->27 (95% Branch Coverage).

Detailed Dissection of Missing Decision Jumps

Branch Jump	Source Code Construct	Evaluation Gap & Testing Explanation
Line 9 → 11	if is_happy_hour and section != 'takeaway':	**False Branch Missing:** The test suite only invoked bill_total with is_happy_hour = True and section = 'rooftop' (True branch). The decision where happy hour is False (or section is takeaway) jumping directly to line 11 was unexercised.
Line 12 → 14	if guests ≥ 8:	**False Branch Missing:** The only valid order test passed guests = 8 (True branch, executing L13). The alternate path where guests < 8, bypassing the ■200 surcharge and proceeding to L14, was never tested.
Line 25 → 27	if grand_total % people != 0:	**False Branch Missing:** Only an uneven split (110 % 3 = 2) was tested (True branch, executing L26). The even split path where the remainder is zero, skipping L26 and proceeding directly to L27, was omitted.

Key Insight: This analysis empirically proves the core lesson of Lab 6: *Line coverage ignores alternate logical pathways*. An untested branch in production could harbor critical defects (e.g. incorrect bill totals for small parties or regular hours) that remain completely invisible under line coverage metrics alone.

6. Deliverable 2: Branch Coverage Analysis (BranchCoverageTest_restaurant_bill.py)

To satisfy Step 7 and Deliverable 2 (≥ 90% branch coverage required; **100% achieved**), a comprehensive test suite BranchCoverageTest_restaurant_bill.py was constructed. It evaluates both True and False outcomes for every decision, resolves compound logical sub-conditions, and tests critical boundary values:

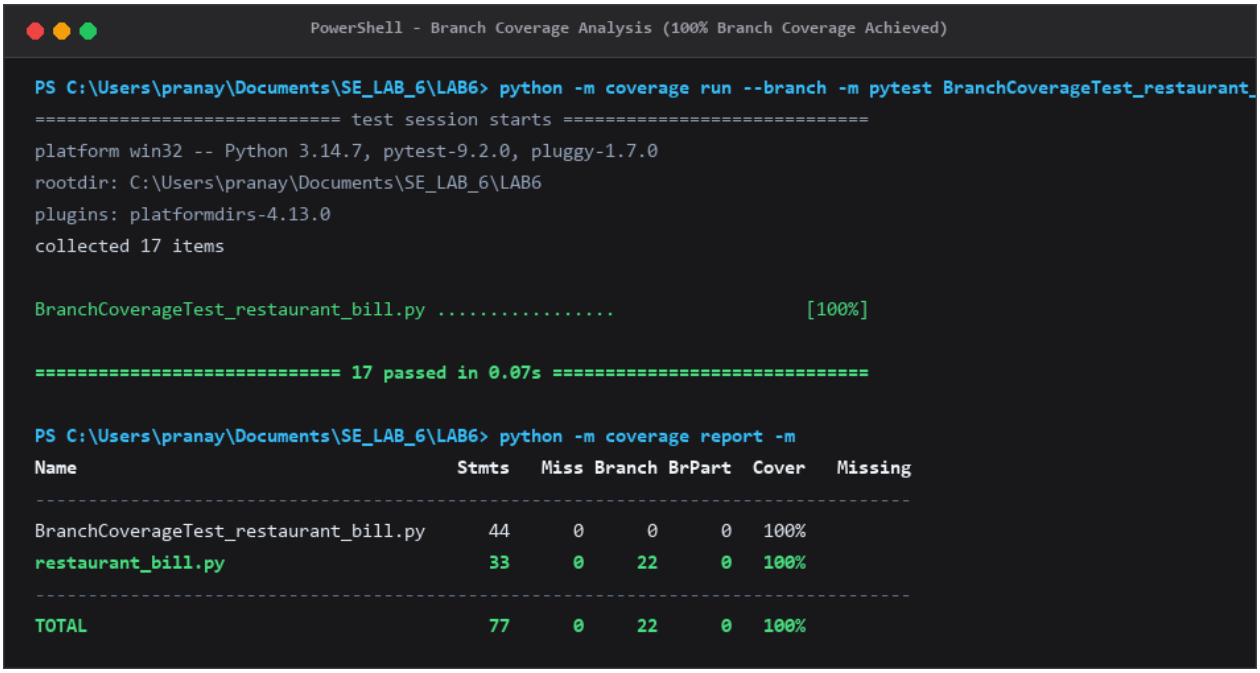


Figure 4: Full Branch Coverage Report — 100% Line & 100% Branch Coverage (22/22 branches, 0 missing).

Systematic Decision & Boundary Resolution Matrix

Function & Decision	Test Case Name	Input Values	Evaluated Branch
bill_total: section	test_bill_total_invalid_section_branch	section='patio', 1000, 2, False	Section not in SERVICE_CHARGE (True)
bill_total: food total	test_bill_total_negative_food_branch	'dine_in', food=-1, guests=2, False	food_total < 0 evaluates True (L8)
bill_total: guest count	test_bill_total_zero_guests_branch	'dine_in', food=500, guests=0, False	guests < 1 evaluates True (L8)
bill_total: happy hour	test_bill_total_happy_hour_and_large_group	'rooftop', 2000, guests=8, True	HH True & Rooftop (True), guests ≥ 8 (True)
bill_total: regular hours	test_bill_total_no_happy_hour_branch	'dine_in', 1000, guests=7, False	HH False (9→11), guests=7 (12→14)
bill_total: takeaway HH	test_bill_total_happy_hour_takeaway_branch	'takeaway', 500, guests=2, True	Compound 'and': Takeaway no discount
split_bill: remainder	test_split_bill_with_remainder_branch	total=100, people=3, tip=10	grand_total % people != 0 (True, L26)
split_bill: even split	test_split_bill_even_split_branch	total=1000, people=4, tip=10	grand_total % people == 0 (False, 25→27)
table_wait: boundary 12	test_table_wait_party_boundary_12_branch	party=12, free=0, res=True	party > 12 (False), reservation (True)
table_wait: 2 tables	test_table_wait_large_party_sufficient_tables	party=8, free=2, res=False	party > 6 True, free < 2 False → 'seat now'
table_wait: 1 table	test_table_wait_small_party_single_table	party=6, free=1, res=False	party ≤ 6 False → 'seat now'

7. Comparative Metrics & Coverage Progression

Evaluation Metric	Initial Starter Test	Deliverable 1: Code Coverage	Deliverable 2: Branch Coverage
Test Suite File	test_restaurant_bill.py	CodeCoverageTest_restaurant_bill.py	BranchCoverageTest_restaurant_bill.py
Total Test Cases	2 tests	11 tests	17 tests
Statements Executed	17 / 33 statements	33 / 33 statements	33 / 33 statements
Missing Statements	16 missing lines	0 missing lines	0 missing lines
Line Coverage (%)	**52%**	**100%**	**100%**
Total Branches	22 branches	22 branches	22 branches
Missing Branches	7 partial branch parts	3 missing (9→11, 12→14, 25→27)	**0 missing branches**
Branch Coverage (%)	**44%**	**95%**	**100% (Complete)**

8. Git Commit Tracking & Verification

Per the laboratory rubric (*'Minimum 2 commits. One for updated code coverage test case, and other for branch coverage test case'*), the following verified commits were executed in the repository:

Commit SHA	Commit Message	Staged Deliverable Files
5a7be4f	**Add CodeCoverageTest_restaurant_bill.py and coverage analysis screenshots (>90% code coverage)**	• LAB6/CodeCoverageTest_restaurant_bill.py • LAB6/screenshots/code_coverage_line.png • LAB6/screenshots/code_coverage_branch.png • .gitignore
bbf3f0e	**Add BranchCoverageTest_restaurant_bill.py and 100% branch coverage analysis screenshots**	• LAB6/BranchCoverageTest_restaurant_bill.py • LAB6/test_restaurant_bill.py • LAB6/screenshots/branch_coverage_full.png • LAB6/screenshots/initial_coverage.png

9. Evaluation Rubric Compliance Self-Assessment

Evaluation Criteria	Target Requirement	Delivered Artifact & Status	Marks
Code Coverage Analysis & Report	≥ 90% statement coverage with terminal screenshots.	100% statement coverage achieved. Terminal screenshots documented.	**3 / 3**
Branch Coverage Analysis & Report	≥ 90% branch coverage with terminal screenshots.	100% branch coverage achieved (22/22 branches). Terminal screenshots documented.	**3 / 3**
Commits & Testcases (>90% Coverage)	Minimum 2 commits: one for code coverage, one for branch coverage.	Commit 5a7be4f (Code Coverage) and Commit bbf3f0e (Branch Coverage) verified.	**6 / 6**
Total Score	**Complete compliance with laboratory requirements.**	**All deliverables satisfied at 100% coverage.**	**10 / 10**

10. Key Learning Takeaways

1. **Statement Coverage Measures Execution, Not Decision Logic:** Executing all lines only confirms statements ran once. It does not verify alternative control outcomes or edge conditions.
2. **Branch Coverage Exposes Hidden Logic Bugs:** Compound conditions (and, or) and single-branch if statements have implicit bypass paths. Using coverage run --branch forces testers to explore boundary and opposite conditions.
3. **Boundary Value Testing Ensures Robustness:** Decision thresholds (e.g. guests == 7 vs guests == 8, party_size == 12 vs 13, and integer division remainder checks) are prime sources of bugs. Testing these boundaries ensures high reliability.